# Typed Symbolic Regions and Resumable CTL

Mettapedia Working Notes

1 November 2026

**Abstract**

Ten Lean modules connect guarded finite-state programs to symbolic Boolean operations, independent temporal path semantics, a resumable fixed-point solver and a shared decision-table evaluator. The forward-image and universal predecessor operators satisfy an adjunction: a requested output predicate determines the largest domain-qualified input set that guarantees output safety. This is an executable, restricted refinement-inference mechanism, not a classifier for arbitrary programs. A separate private-fold theorem preserves ordinary results through late refusal while keeping interruption distinct. The results expose the information that an implementation must retain: a shared source witness, valid-state domain, fixed transition meaning, approximation polarity and original fallback state.

## 1 Scope and representation

The symbolic state carrier is $\Sigma_n = \mathrm{Fin}(n) \to \mathrm{Bool}$. A diagram is an existing `BDDCore.BDDn` paired with its ordered and reduced invariant. Boolean operations reuse the existing BDD implementation. No second Boolean decision-tree datatype is introduced for the logical layer.

A source rule consists of a pure Boolean guard and one Boolean expression for each next-state bit. All expressions read the same old state. A list of rules denotes a nondeterministic relation: a transition exists when some enabled rule produces its target state. A separate list observer retains repeated rule occurrences; the relational observer intentionally forgets them.

The valid encoded states form an explicit diagram $D$. The restricted relation $R_D$ requires both endpoints to lie in $D$. The executable rule checker tests whether

$$D(s) \wedge g(s) \wedge \neg D(\mathrm{next}(s))$$

is unsatisfiable. Its equivalence with semantic invariant preservation is proved. Successful inference returns a program carrying a proof that every rule preserves $D$. Consequently, restricting to $D$ drops no transition from a valid source state. Without that certificate, a domain restriction could conceal an escaping transition rather than justify an optimization.

This is a source language of guarded bit-vector assignments, not yet a decoder for arbitrary MeTTa atoms or the CTL benchmark's state identifiers. The relevant executable and proof interfaces are `StateTransducer.infer`, `checkRule_iff` and `valid_transition_not_dropped`.

## 2 Substitution and correlated transition images

The substitution compiler replaces each diagram variable with a Boolean diagram. Its theorem is simultaneous substitution:

$$[\![\mathrm{substitute}(\sigma, t)]\!](s) = [\![t]\!](v \mapsto [\![\sigma(v)]\!](s)).$$

This permits a symbolic predecessor to compute the effect of an entire assignment vector without sequentially mutating its inputs. The swap control distinguishes this operation from in-place assignment.

For a forward image, the transition graph uses an output-bit block followed by an input-bit block. Coordinate constraints are conjoined before the input block is existentially projected away. Thus every output coordinate shares *one* source witness. For a rule $(x, y) \mapsto (x, x)$, the reachable outputs satisfy equality of the two coordinates. Projecting the coordinates independently would admit incompatible outputs such as $(1, 0)$; a checked negative control excludes that mistake.

For diagrams $A, B$, the symbolic operations implement

$$\begin{aligned} \mathrm{post}_{R_D}(A) &= \{t \mid \exists s \in A,\ R_D(s,t)\}, \\ \mathrm{must}_{R_D}(B) &= \{s \in D \mid \forall t,\ R_D(s,t) \Rightarrow t \in B\}. \end{aligned}$$

The forward operator uses the existing existential-projection machinery. The backward operator is a domain-relative complement of an existential predecessor. Global negation is insufficient when not every bit pattern is a valid encoded state.

## 3 Output safety as refinement inference

`RelationalImage.image_must_adjunction` establishes

$$\mathrm{post}_{R_D}(A) \subseteq B \quad \Longleftrightarrow \quad A \cap D \subseteq \mathrm{must}_{R_D}(B).$$

The right adjoint is therefore the largest domain-qualified set of inputs whose successors satisfy the requested output refinement. This moves beyond recognizing one syntactic spelling: different rule presentations with the same transition relation induce the same semantic safety condition.

The scope of "largest" matters. It is exact for this finite-state relation and this output observation, not globally optimal code, minimum description length or a cheapest storage representation. The construction can produce large diagrams. A profitability decision needs a separate cost model.

Output safety does not imply progress. A state with no successor satisfies every universal output condition vacuously, including the false condition. The negative control computes that case and contrasts it with a false existential-successor test. A profile requiring a result must separately require existence; an infinite-run requirement is stronger still.

The information boundary is precise: existential images and CTL state sets may forget occurrence multiplicity, but a bag-valued source observer may not. Likewise, source correlations, valid-state membership and transition meaning remain necessary even when the physical representation becomes smaller.

## 4 Independent temporal semantics

The temporal reference meaning is not defined by the symbolic solver. `CTLFixedPoints.Until` is an inductive finite path: a state either already meets the goal, or satisfies the holding condition and takes a step to another such path. `Globally` is an infinite sequence of states with the holding predicate and transition relation satisfied at every index. The formalization proves

$$\begin{aligned} E[H\ U\ G] &= \mu X.\ G \cup (H \cap \mathrm{pre}_R(X)), \\ EG\,H &= \nu X.\ H \cap \mathrm{pre}_R(X). \end{aligned}$$

The reverse greatest-fixed-point direction constructs an infinite path by choosing successors inside the greatest fixed point. Its use of classical choice is visible in the foundation audit.

Deadlocks are explicit. A deadlocked state cannot satisfy the infinite-path meaning of $EG\,H$, even when $H$ holds there. Adding a self-loop changes that answer. The typed transducer supplies an explicit stuttering completion and proves that every valid state then has a valid successor. The finite-until reference also permits the goal to hold immediately without a successor. Connecting to a total-Kripke CTL profile must therefore select its deadlock policy rather than silently changing the source relation.

The symbolic implementations of predecessor, until and globally refine these independent meanings. Nested temporal-formula parsing, source-state codecs, fairness constraints and a whole-formula compilation pipeline remain separate interfaces. The classic symbolic model-checking construction already uses Boolean representations and least/greatest fixed points [1]; the contribution here is its checked integration with typed admission, resumable observations and the repository's existing carriers.

## 5 Resumption and the information in a partial answer

A symbolic operator carries a local refinement to an independent monotone set operator. The solver starts at bottom for a least fixed point and at top for a greatest fixed point. Each application increments a pass counter and uses a complete diagram-equivalence test. Equivalence is implemented by XOR and a checked zero test; no constant-time pointer-equality claim is made.

The result type distinguishes `pausedframe` from `doneframe`. For a fixed operator, the page law is exact:

$$\mathrm{resume}(b, \mathrm{advance}(a, f)) = \mathrm{advance}(a + b, f).$$

It preserves the diagram, completion status and solver pass count, including the final stability check. These passes are not MeTTa evaluator instructions or the source benchmark's round counter.

Completion implies a fixed point. Bottom and top invariants identify the completed value as the least and greatest fixed point respectively. Finite state spaces provide ascending and descending chain conditions, so there exists sufficient fuel for completion. This is an eventual-completion theorem, not a numeric bound on runtime or on the cost of one Boolean operation.

Partial answers have opposite information polarity:

| Iteration | Already certified | Still unresolved before completion |
|---|---|---|
| Least fixed point | States already included | States not yet included |
| Greatest fixed point | States already excluded | States still retained |

The two observation functions return `OptionBool` and prove soundness of every returned Boolean. At zero fuel, an empty least approximation and a full greatest approximation both yield unresolved observations where their raw bits alone would be misleading.

The operator and its domain remain fixed across pages. A control shows that changing the relation can invalidate a completed answer. A native task must therefore retain a captured relation or qualify reuse against its revision; the page law alone is not a concurrent-mutation theorem.

## 6 A shared decision table with independent evaluation

`DecisionTable` stores constants and tests whose two child indices refer to earlier rows. Its direct evaluator caches one Boolean per row. An independent decoder expands the records into existing BDD trees; the main theorem proves equality between direct evaluation and the decoded tree semantics. Invalid references return `none`, not false, including an invalid child on a branch that the current input would not select.

Append-only evaluation preserves old indexed values. The counted evaluator visits exactly one record per row on successful evaluation. A nontrivial diamond contains six records but unfolds to eleven tree nodes. This is a record-visit result, not a linear CPU-time result for the list-based reference implementation: append and index lookup costs are not erased. Nor does diagram node count measure represented state count; a one-record true diagram represents all eight assignments to three bits.

The table is a real shared carrier, but the current package does not yet compile arbitrary BDD trees into hash-consed tables, run memoized Boolean operations over those tables, or verify native collection and relocation. Those are the next representation refinements.

## 7 Private fold fallback and interruption

`Machines.PrivateScalarFold` reuses the existing source fold carrier. A native step can return a private next accumulator or refuse. The local refinement requires every accepted step to equal one pure, single-answer source callback at the captured world. Induction then proves equality for an entirely accepted fold, including its unchanged world.

If a later step refuses, the dispatcher runs the ordinary source fold from the original accumulator and world, over the original ready items. It does not continue from a partially updated native accumulator. An interruption has a separate outcome and never initiates ordinary replay. The controls include negative arithmetic, late signed-range overflow, cancellation with an effectful fallback and a refuted purity obligation.

The signed-range control uses exact integers with an explicit result bound $[-2^{63}, 2^{63})$; it is not a proof of an Int64 ABI or of the runtime's operand-admission checks. The general theorem starts after item and initializer production. It does not permit replaying their effects. Actual signed/symbol operations, conditional and tuple extraction, callability checks, program revisions, arenas and publication still need source and native bridges.

For lifetime design, this identifies a concrete obligation: the original fallback inputs must remain available until the attempt's disposition is settled. Private progress does not make them unreachable merely because it has produced a newer accumulator.

## 8 Checked modules and remaining interfaces

The ten mathematical modules are:

- `Algebra.Order.FixedPointIteration` and `Logic.CTLFixedPoints`;
- `Logic.BDD.StatePredicates`, `StateTransducer`, `RelationalImage`, `FixedPointSolver`, `SymbolicCTL`, `DecisionTable` and `SymbolicCTLControls`;
- `Machines.PrivateScalarFold`.

`Logic.BDD.SymbolicRegionAudit` imports the package, prints the main theorems' axiom dependencies and emits declaration-level foundation manifests. The audit checks 563 declaration bodies, with ten nonlogical recursion companions reported separately. There are no admissions or added axioms; transitive primitive dependencies are confined to `propext`, `Classical.choice` and `Quot.sound`. Strict file checks treat warnings as errors. This is a targeted package audit, not a whole-repository audit.

The immediate source task remains certified extraction of the actual scalar fold, including its signed arithmetic, inert symbols, branches and flat tuples. The symbolic task needs source codecs and formula compilation before it can stand in for a source CTL computation. Native layouts, revision checks, allocation failure and measured plan choice then refine the two routes without changing their observation contracts.

## References

[1] J. R. Burch, E. M. Clarke, K. L. McMillan, D. L. Dill and L. J. Hwang. *Symbolic Model Checking:* $10^{20}$ *States and Beyond*. Information and Computation 98(2), 142–170, 1992. doi:10.1016/0890-5401(92)90017-A. Author-hosted full text.